AI 协作日志

空洞文件「断点续传」下载器 · 需求理解与实现过程记录

项目：空洞文件「断点续传」下载器（鲁斑猫 RK3566）

协作方式：Claude Code+deepseek（AI 编程助手）

交付内容：main.c · Makefile · README.md · AI 协作日志

日期：2026 年 10 月 14 日

——电信 3 班任俊杰（202434310321）

1. 需求理解过程

轮次	我提出的问题 / 说明	AI 的澄清与反馈
1	只给目录名「IO 空洞文件」，让 AI 先分析	AI 判断为「文件空洞 / 稀疏文件」主题，并提示平台差异：Linux 用 lseek 造空洞，Windows 需 Win32 稀疏文件 API
2	说明是作业	AI 追问目标平台与题目要求
3	给出题目：用文件 I/O 制作空洞文件（≥128KB），命令行传参实时修改（类似下载）	AI 确认三点要求，并给出命令设计草案
4	补充：鲁斑猫 RK3566、断点续传记录「已完成偏移」、Ctrl+C 续传、status 打印三指标、交付源码+Makefile+README+AI 协作日志	AI 形成完整方案并进入规划模式，经确认后实现

2. 关键决策记录

- **平台：**Linux（鲁斑猫 RK3566，ARM64），采用 POSIX open/lseek/write/stat 写法；Makefile 用 CC ?= gcc 支持板载编译与 aarch64-linux-gnu-gcc 交叉编译。
- **下载数据来源：**题目为「类似下载功能」且未提供数据源，故 download 写入确定性字节模式 (offset + i) % 256 模拟下载数据，便于验证定点写入结果。
- **断点记录方式：**旁路文件 <file>.progress，文本保存「已完成偏移」（即下次写入起点）。
- **Ctrl+C 处理：**signal(SIGINT, ...) 置 volatile sig_atomic_t 标志，循环在块边界保存断点后干净退出。
- **空洞占比公式：**(1 - st_blocks*512 / st_size) × 100%。

3. 分工

- **AI 完成：**源码 main.c、Makefile、README.md、本协作日志的框架与决策记录。
- **本人完成：**在鲁斑猫上的编译、运行、验证与截图。

4. 实测记录

环境：鲁斑猫 RK3566（aarch64）Linux，编译产物 hole_download。

编译：`gcc -Wall -Wextra -O2 -o hole_download main.c` 一次通过，无警告无错误。

① 创建空洞文件（create）

```
已创建稀疏文件 test.bin，逻辑大小 131072 字节 (128.00 KB)，断点初始化为 0
```

② 初始空洞占比 ≈ 100%

新建 1GB 大文件后执行 status，空洞占比接近 100%。128KB 文件太小，使用大文件验证更直观。

③ 下载写入（download）

分块逐块写入，进度从 0% 增长到 100%。

④ 断点续传（Ctrl+C）

使用 1GB 大文件，download 中途按 Ctrl+C，打印「收到 Ctrl+C, 断点已保存: xxx 字节」；再次 download 显示「从断点 xxx 继续」，从断点续传成功。

⑤ 完成识别

重复 download 输出「文件 test.bin 已下载完成(断点 131072 == 总大小 131072)」。

⑥ 下载完成后空洞占比 = 0%

```
文件: test.bin
逻辑大小:      131072 字节 (128.00 KB)
实际占用块数: 256 块 (实际占用 131072 字节)
空洞占比:      0.00%
```

⑦ 定点写入（write）

```
已在 test.bin 偏移 100 处定点写入 5 字节: "hello"
```

配合 od -c 查看偏移 100 附近，可见 hello 前后字节未被破坏，证明定点写入不影响其余空间。

5. 遇到的问题与解决

1. **目录多一层导致编译报错：**文件整包上传后实际路径为 ~/Desktop/Ke-Cheng-She-Ji/IO 空洞文件/，若在 Ke-Cheng-She-Ji 下编译会报 gcc: error: main.c: No such file or directory。 解决：cd ~/Desktop/Ke-Cheng-She-Ji/io* 进入正确目录后再编译。

128KB 太小、无法演示 Ctrl+C 断点：128KB 下载瞬间完成，来不及按 Ctrl+C。 解决：改用 1GB 大文件（create big.bin 1073741824）放慢进度后再中断。

实验步骤

空洞文件断点续传下载器

项目目录： `~/Desktop/Ke-Cheng-She-Ji/io 空洞文件`

运行环境： 板子项目目录（所有命令均在项目目录下执行）

可执行文件： `hole_download`（由 `main.c` 编译生成）

〇、编译准备

首次编译（在项目目录下）：

```
cd ~/Desktop/Ke-Cheng-She-Ji/io 空洞文件

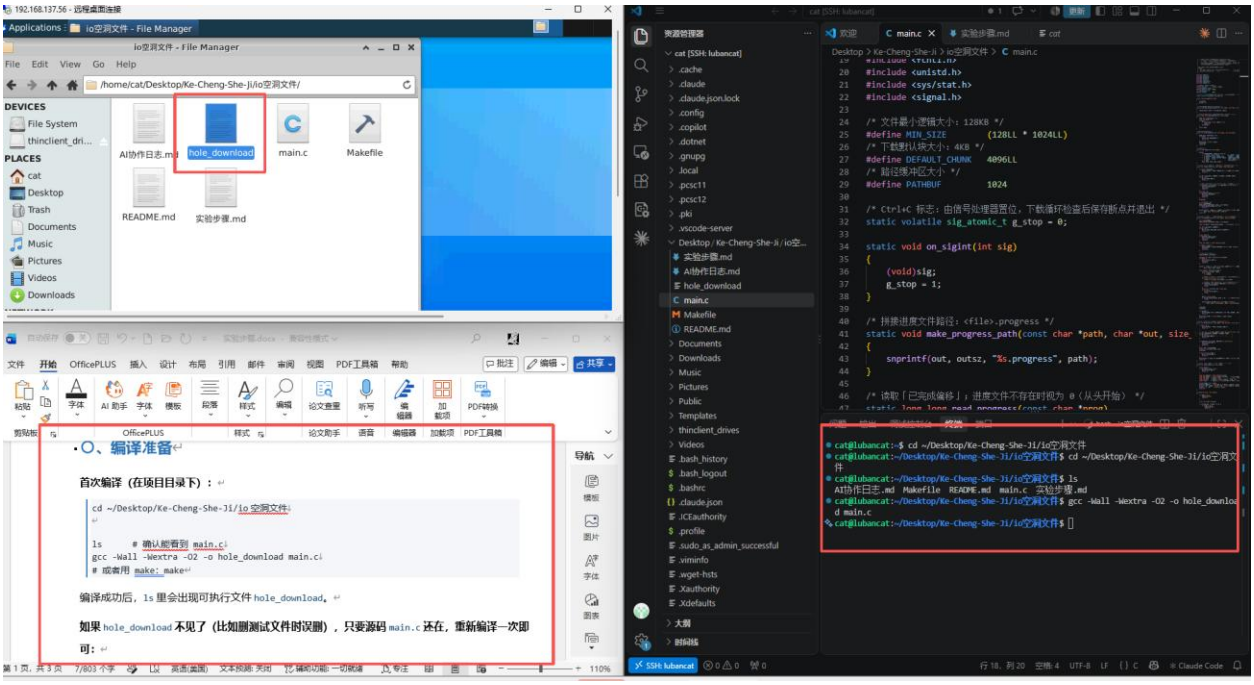

ls      # 确认能看到 main.c
gcc -Wall -Wextra -O2 -o hole_download main.c
# 或者用 make：make
```

编译成功后，`ls` 里会出现可执行文件 `hole_download`。

如果 `hole_download` 不见了（比如删测试文件时误删），只要源码 `main.c` 还在，重新编译一次即可：

```
ls
gcc -Wall -Wextra -O2 -o hole_download main.c
```

如果连 `main.c` 都没有了（源码也被删），需从 Windows 重新上传源码，再执行上面的编译。

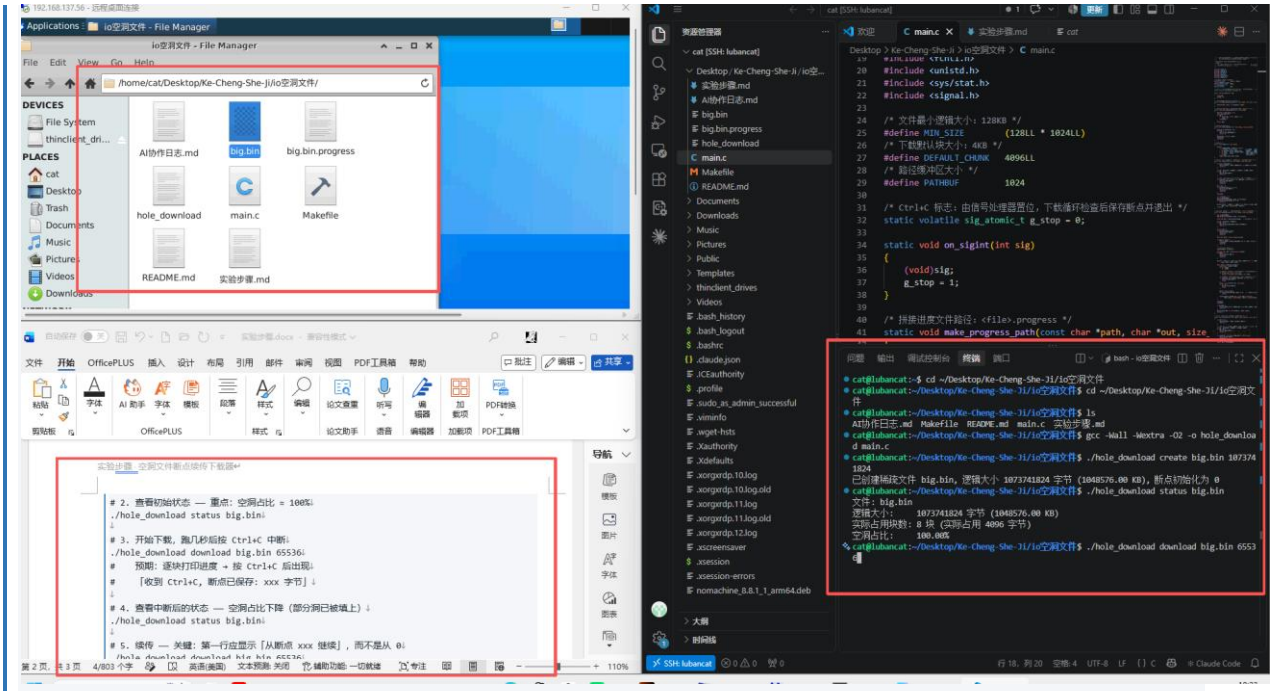


一、核心功能演示（断点续传）

> 用大文件（1GB）演示，因为 128KB 太小、下载瞬间完成，来不及按 Ctrl+C。

```
# 1. 创建 1GB 稀疏文件（瞬间完成，因为中间全是空洞）
./hole_download create big.bin 1073741824

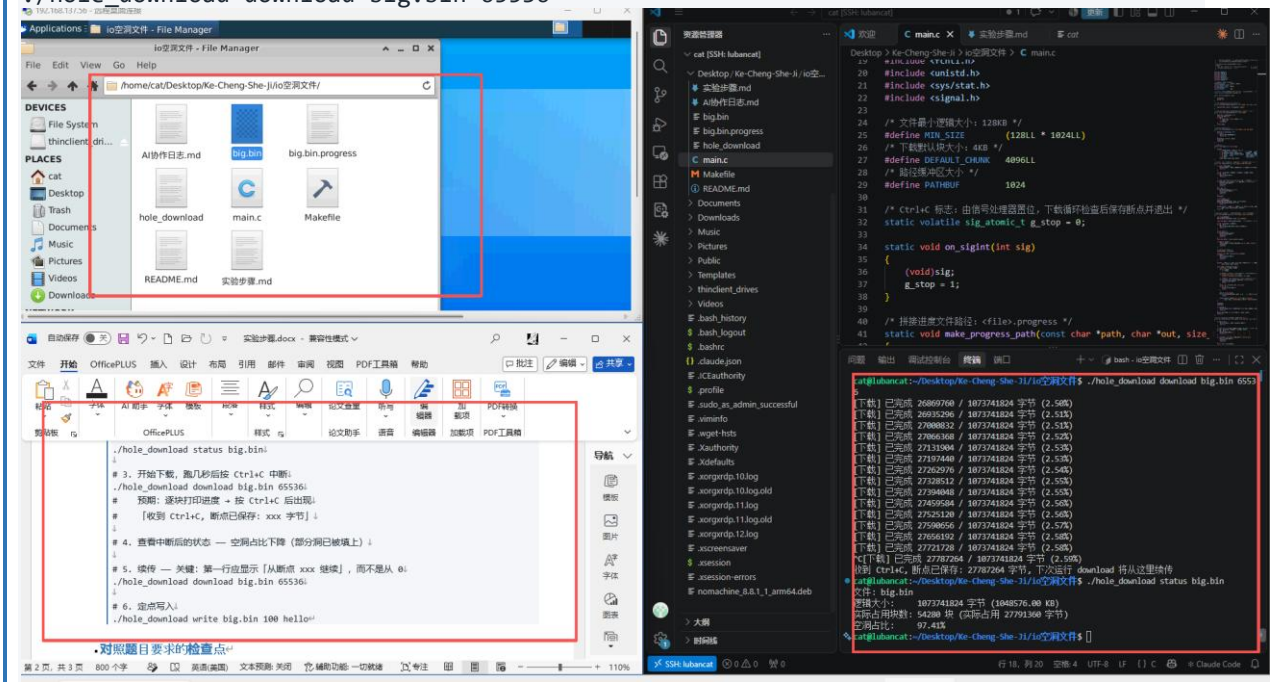
# 2. 查看初始状态 — 重点：空洞占比 ≈ 100%
./hole_download status big.bin
```



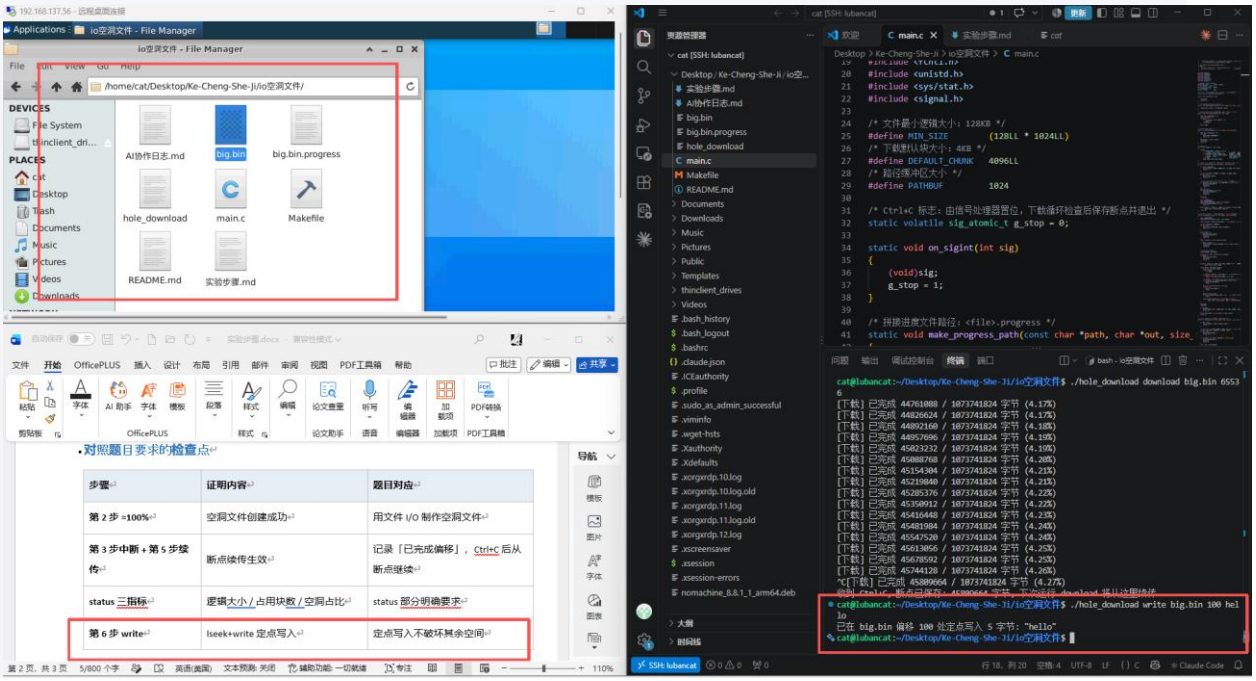
```
# 3. 开始下载，跑几秒后按 Ctrl+C 中断
./hole_download download big.bin 65536
#     预期：逐块打印进度 → 按 Ctrl+C 后出现
#     「收到 Ctrl+C，断点已保存：xxx 字节」

# 4. 查看中断后的状态 — 空洞占比下降（部分洞已被填上）

# 5. 续传 — 关键：第一行应显示「从断点 xxx 继续」，而不是从 0
./hole_download download big.bin 65536
```



```
# 6. 定点写入
./hole_download write big.bin 100 hello
```



对照题目要求的检查点

步骤	证明内容	题目对应
第 2 步 ≈100%	空洞文件创建成功	用文件 I/O 制作空洞文件
第 3 步中断 + 第 5 步续传	断点续传生效	记录「已完成偏移」，Ctrl+C 后从断点继续
status 三指标	逻辑大小 / 占用块数 / 空洞占比	status 部分明确要求
第 6 步 write	lseek+write 定点写入	定点写入不破坏其余空间

二、特色创新项

A. 空洞不占磁盘空间（「空洞」的核心价值）

新建文件后、下载前立即执行（此时为全洞状态）：

```
./hole_download create hole.bin 1073741824
ls -l  hole.bin     # 逻辑大小：1073741824 (1GB)
du -h  hole.bin     # 实际磁盘占用：几乎为 0 (如 4.0K)
```

结论：逻辑 1GB，物理几乎 0，这就是空洞文件的本质。

B. 空洞区域读出来是全 0

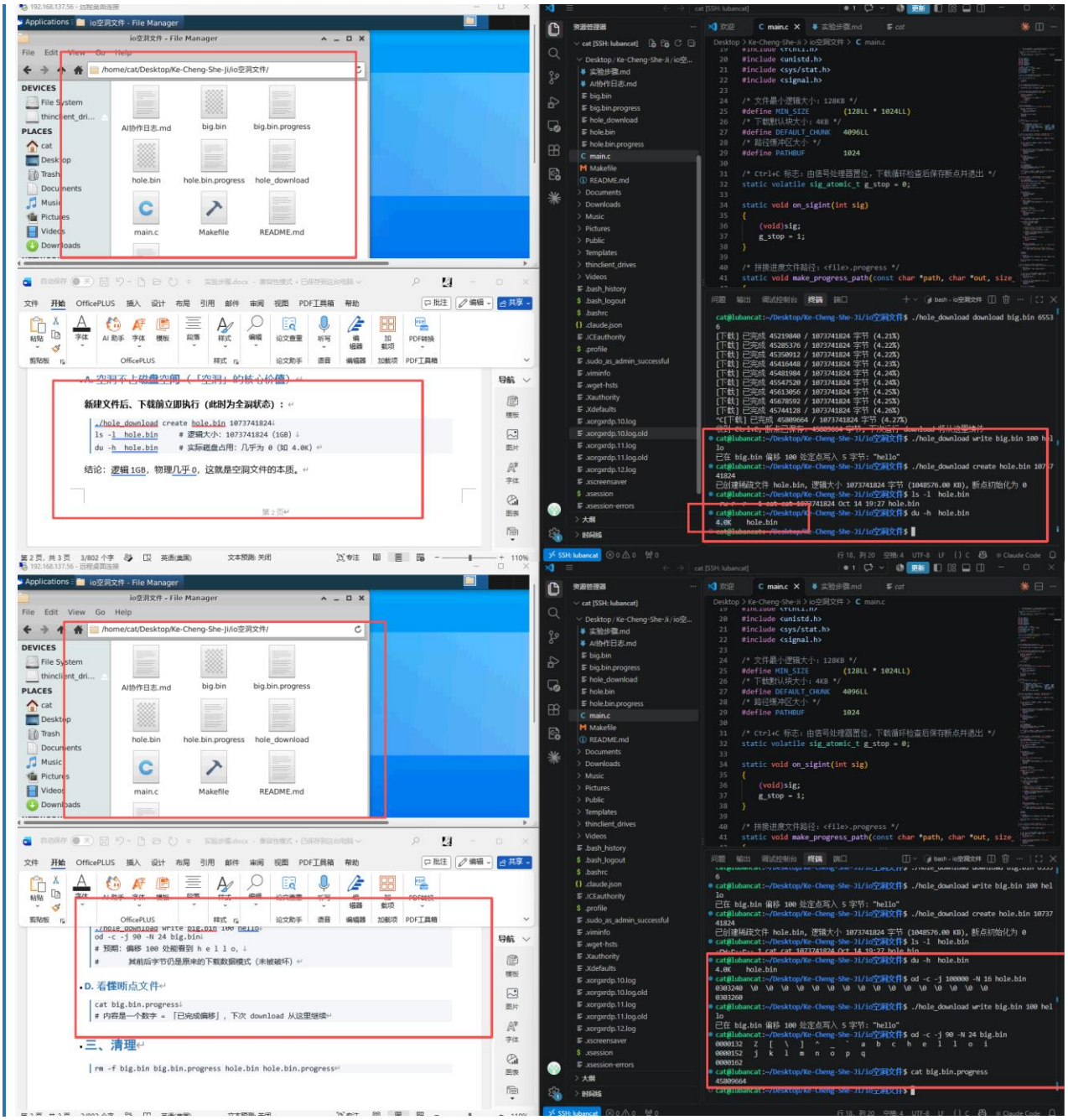
```
od -c -j 100000 -N 16 hole.bin
# 预期：全是 \0 \0 \0 ... (尚未下载到的区域)
```

C. 验证定点写入只改目标位置、不破坏别处

```
./hole_download write big.bin 100 hello
od -c -j 90 -N 24 big.bin
# 预期：偏移 100 处能看到 h e l l o,
#       其前后字节仍是原来的下载数据模式（未被破坏）
```

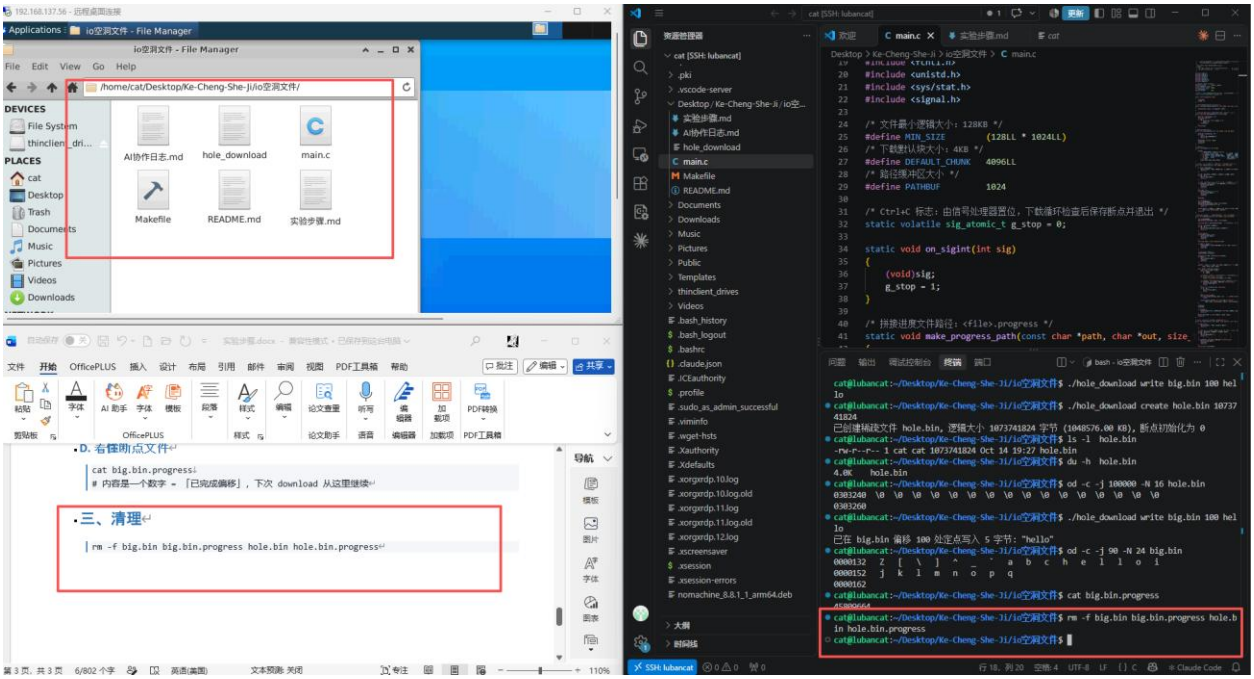
D. 看懂断点文件

```
cat big.bin.progress
# 内容是一个数字 = 「已完成偏移」，下次 download 从这里继续
```



三、清理

```
rm -f big.bin big.bin.progress hole.bin hole.bin.progress
```


io空洞文件 - File Manager
/home/cat/Desktop/Ke-Cheng-She-Ji/io空洞文件/
AI协作日志.md
hole_download
main.c
Makefile
README.md
实验步骤.md
三、清理
rm -f big.bin big.bin.progress hole.bin hole.bin.progress
cat big.bin.progress
内容是一个数字 - 「已完成偏移」，下次 download 从这里继续
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ ./hole_download write big.bin 100 hello
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ ./hole_download create hole.bin 1073741824
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ ls -l hole.bin
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ du -h hole.bin
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ od -c -j 100000 -N 16 hole.bin
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ ./hole_download write big.bin 100 hello
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ od -c -j 90 -N 24 big.bin
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ cat big.bin.progress
cat@Lubancat:~/Desktop/Ke-Cheng-She-Ji/io空洞文件$ rm -f big.bin big.bin.progress hole.bin hole.bin.progress

附录：

源代码：

```
/*
 * 空洞文件「断点续传」下载器
 *
 * 基于文件 I/O 的空洞文件（sparse file）演示程序：
 *   - 用 lseek + write 在固定偏移定点写入，制造/填充文件空洞
 *   - 通过命令行传参控制：create / download / write / status / help
 *   - download 支持断点续传：进度（已完成偏移）记录在 <file>.progress，
 *     Ctrl+C（SIGINT）后保存断点，下次运行从断点继续
 *
 * 运行平台：鲁班猫 RK3566（ARM64 Linux）
 * 编译：
 *   gcc -Wall -Wextra -O2 -o hole_download main.c     # 板载原生编译
 *   aarch64-linux-gnu-gcc ... 同上                    # PC 交叉编译
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
```

```
#include <signal.h>


/* 文件最小逻辑大小：128KB */
#define MIN_SIZE       (128LL * 1024LL)
/* 下载默认块大小：4KB */
#define DEFAULT_CHUNK   4096LL
/* 路径缓冲区大小 */
#define PATHBUF        1024


/* Ctrl+C 标志：由信号处理器置位，下载循环检查后保存断点并退出 */
static volatile sig_atomic_t g_stop = 0;


static void on_sigint(int sig)
{
    (void)sig;
    g_stop = 1;
}


/* 拼接进度文件路径：<file>.progress */
static void make_progress_path(const char *path, char *out, size_t outsz)
{
    snprintf(out, outsz, "%s.progress", path);
}
```

```
/* 读取「已完成偏移」；进度文件不存在时视为 0（从头开始） */
static long long read_progress(const char *prog)
{
    FILE *fp = fopen(prog, "r");
    long long done = 0;
    if (fp) {
        if (fscanf(fp, "%lld", &done) != 1)
            done = 0;
        fclose(fp);
    }
    return done;
}

/* 写入「已完成偏移」 */
static int write_progress(const char *prog, long long done)
{
    FILE *fp = fopen(prog, "w");
    if (!fp) {
        perror("写进度文件失败");
        return -1;
    }
    fprintf(fp, "%lld\n", done);
    fclose(fp);
    return 0;
```

```
}


/* 打印用法 */
static void usage(const char *prog)
{
    printf(
        "空洞文件「断点续传」下载器 用法:\n"
        "  %s create  <file> <size>          建 >=128KB 稀疏文件(全洞), 初始化断点=0\n"
        "  %s download <file> [chunk_size]     从断点续传, 分块定点写入填洞; Ctrl+C 保存断点\n"
        "  %s write    <file> <offset> <data>   lseek+write 定点写入(不破坏其他空间)\n"
        "  %s status   <file>                打印 逻辑大小/实际占用块数/空洞占比\n"
        "  %s help                           显示本帮助\n",
        prog, prog, prog, prog, prog);
}


/* create: 创建 >=128KB 的稀疏文件(全洞), 并把断点初始化为 0 */
static int create_file(const char *path, long long size)
{
    if (size < MIN_SIZE) {
        fprintf(stderr, "错误: 文件大小必须 >= 128KB (%lld 字节)\n", MIN_SIZE);
        return 1;
    }
```

```
int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) {
    perror("open 失败");
    return 1;
}

/* 关键: 把偏移跳到 size-1 处再写 1 字节, 中间区域即为空洞 */
if (lseek(fd, (off_t)(size - 1), SEEK_SET) == (off_t)-1) {
    perror("lseek 失败");
    close(fd);
    return 1;
}
char zero = 0;
if (write(fd, &zero, 1) != 1) {
    perror("write 失败");
    close(fd);
    return 1;
}
close(fd);

/* 初始化断点 = 0 */
char prog[PATHBUF];
make_progress_path(path, prog, sizeof(prog));
if (write_progress(prog, 0) != 0)
```

```
        return 1;

    printf("已创建稀疏文件 %s, 逻辑大小 %lld 字节 (%.2f KB), 断点初始化为 0\n",
           path, size, (double)size / 1024.0);
    return 0;
}

/* download: 从断点续传, 分块定点写入填洞; Ctrl+C 保存断点后退出 */
static int download_file(const char *path, long long chunk)
{
    char prog[PATHBUF];
    make_progress_path(path, prog, sizeof(prog));

    long long done = read_progress(prog);   /* 已完成偏移 = 下一次写入起点 */

    int fd = open(path, O_WRONLY);
    if (fd < 0) {
        perror("open 失败(请先 create 创建文件)");
        return 1;
    }

    struct stat st;
    if (fstat(fd, &st) != 0) {
        perror("fstat 失败");
```

```
    close(fd);
    return 1;
}
long long total = (long long)st.st_size;

if (done >= total) {
    printf("文件 %s 已下载完成(断点 %lld == 总大小 %lld)\n", path, done, total);
    close(fd);
    return 0;
}

/* 安装 Ctrl+C 处理器 */
signal(SIGINT, on_sigint);

unsigned char *buf = malloc((size_t)chunk);
if (!buf) {
    perror("malloc 失败");
    close(fd);
    return 1;
}

printf("开始下载 %s: 总大小 %lld 字节, 从断点 %lld 继续, 块大小 %lld\n",
       path, total, done, chunk);
```

```
while (done < total && !g_stop) {
    long long n = total - done;
    if (n > chunk)
        n = chunk;

    /* 填充确定性字节模式, 模拟下载数据(便于验证定点写入结果) */
    for (long long i = 0; i < n; i++)
        buf[i] = (unsigned char)((done + i) % 256);

    /* 定点写入: lseek 到固定偏移再 write, 不影响其他空间 */
    if (lseek(fd, (off_t)done, SEEK_SET) == (off_t)-1) {
        perror("lseek 失败");
        break;
    }
    ssize_t w = write(fd, buf, (size_t)n);
    if (w < 0) {
        perror("write 失败");
        break;
    }

    done += (long long)w;
    if (write_progress(prog, done) != 0)    /* 每块写完后保存断点 */
        break;
```

```
        printf("[下载] 已完成 %lld / %lld 字节 (%.2f%%)\n",
               done, total, (double)done * 100.0 / (double)total);
    }

    free(buf);
    close(fd);

    if (g_stop) {
        printf("收到 Ctrl+C, 断点已保存: %lld 字节, 下次运行 download 将从这里续传\n", done);
        return 0;
    }
    printf("下载完成: %s (%lld 字节)\n", path, total);
    return 0;
}

/* write: lseek+write 定点写入, 不破坏其余空间 */
static int write_at(const char *path, long long off, const char *data)
{
    if (off < 0) {
        fprintf(stderr, "错误: 偏移不能为负\n");
        return 1;
    }

    /* 注意: 不使用 O_TRUNC, 避免清空文件; 先 lseek 再 write 实现定点写入 */
```

```
    int fd = open(path, O_WRONLY | O_CREAT, 0644);
    if (fd < 0) {
        perror("open 失败");
        return 1;
    }

    if (lseek(fd, (off_t)off, SEEK_SET) == (off_t)-1) {
        perror("lseek 失败");
        close(fd);
        return 1;
    }
    size_t len = strlen(data);
    ssize_t w = write(fd, data, len);
    close(fd);

    if (w != (ssize_t)len) {
        perror("write 失败");
        return 1;
    }
    printf("已在 %s 偏移 %lld 处定点写入 %zu 字节: \"%s\"\n", path, off, len, data);
    return 0;
}

/* status: 打印 逻辑大小 / 实际占用块数 / 空洞占比 */
```

```
static int show_status(const char *path)
{
    struct stat st;
    if (stat(path, &st) != 0) {
        perror("stat 失败");
        return 1;
    }

    long long logical = (long long)st.st_size;
    long long blocks  = (long long)st.st_blocks;   /* 512 字节块数 */
    long long used    = blocks * 512;              /* 实际占用字节 */
    long long hole    = logical > used ? logical - used : 0;
    double ratio      = logical > 0 ? (double)hole * 100.0 / (double)logical : 0.0;

    printf("文件: %s\n", path);
    printf("逻辑大小:    %lld 字节 (%.2f KB)\n", logical, (double)logical / 1024.0);
    printf("实际占用块数: %lld 块 (实际占用 %lld 字节)\n", blocks, used);
    printf("空洞占比:    %.2f%%\n", ratio);
    return 0;
}

int main(int argc, char *argv[])
{
```

```
    if (argc < 2) {
        usage(argv[0]);
        return 1;
    }

    const char *cmd = argv[1];

    if (strcmp(cmd, "create") == 0) {
        if (argc != 4) {
            usage(argv[0]);
            return 1;
        }
        long long size = strtoll(argv[3], NULL, 10);
        return create_file(argv[2], size);
    } else if (strcmp(cmd, "download") == 0) {
        if (argc < 3 || argc > 4) {
            usage(argv[0]);
            return 1;
        }
        long long chunk = DEFAULT_CHUNK;
        if (argc == 4) {
            chunk = strtoll(argv[3], NULL, 10);
            if (chunk <= 0)
                chunk = DEFAULT_CHUNK;
        }
```

```
        return download_file(argv[2], chunk);
    } else if (strcmp(cmd, "write") == 0) {
        if (argc != 5) {
            usage(argv[0]);
            return 1;
        }
        long long off = strtoll(argv[3], NULL, 10);
        return write_at(argv[2], off, argv[4]);
    } else if (strcmp(cmd, "status") == 0) {
        if (argc != 3) {
            usage(argv[0]);
            return 1;
        }
        return show_status(argv[2]);
    } else if (strcmp(cmd, "help") == 0 || strcmp(cmd, "--help") == 0 ||
               strcmp(cmd, "-h") == 0) {
        usage(argv[0]);
        return 0;
    }

    usage(argv[0]);
    return 1;
}
```